THANKS TO OUR SPONSORS!

boisecodecamp.com

Celebrating 20 Years
2006 to 2026

COLLEGE OF WESTERN IDAHO

Shooting Trio PHOTOGRAPHY

ZERRTECH

IDAHO TECHNOLOGY COUNCIL

in time tec
CREATING ABUNDANCE

Schedule

8:30	
9:30	Sessions are ~50 minutes
10:30	10 minute until next session
Lunch	90 min lunch *(on your own)*
1:00	Closing panel session is 2 PM
2:00	

Get the App

boisecodecamp.com

BOISE CODE CAMP

BOISE CODE CAMP 2026 · TECHNICAL WORKSHOP

Want AI That Actually Does Things For You?

Building & Running Powerful Personal **AI Agents** Locally with **OpenClaw** and **Hermes Agent** from Nous Research

Presenter: Chuck Anderson

Partially Powered by Nvidia

Date: 5/2/2026]

⚠ Legal Disclaimer

No Warranty or Guarantee

This presentation is provided “as is” without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and non-infringement. The author makes no guarantee that the information contained herein is accurate, complete, or current. Use of any information in this presentation is at your own risk.

Limitation of Liability

In no event shall the author or presenter be held liable for any direct, indirect, incidental, special, or consequential damages arising out of or in connection with the use of this presentation or any information contained within it. This includes, without limitation, damages for loss of profits, data, or other intangible losses.

License — CC BY-NC-ND 4.0

About Today's Session

What to expect from the next 120 minutes

Hands-on focus

Theory + Real demo on actual hardware — not just slides

No prior agent experience required

Beginner-friendly — we'll build up from the basics together

You'll leave with real takeaways

Working examples, practical commands, and **clear next steps** to get started

Agenda & Timeline (120 Minutes)

CONCEPTS → BREAK → DEMO → TIPS → Q&A

0–50 min

Concepts, Architecture, Models, Mercury Agent & SOUL.md

50 MINUTES

50–55 min

Short Break

(5–7 minutes)

5 MINUTES

55–100 min

Live Physical Demo

Setup + Real Tasks

45 MINUTES

100–115 min

Advanced Tips & Best Practices

15 MINUTES

115–120 min

Q&A and Closing

5 MINUTES

Why You Would Want Local AI

PRIVACY FIRST

Your data never has to leave your device

REAL ACTIONS

AI can act — not just talk

PERSISTENT MEMORY

Learns from you and remembers over time

ZERO SUBSCRIPTIONS

No recurring subscription costs

NEW WAY OF THINKING

```
Select * from tblPatient;
```

Vs

Get me a list of Patients.

Results are what matter — tools are just the means.

THE NEW HARSH REALITY OF AI CODE

> The business doesn't care what language your program is written in!

Ship value. Ship fast. Ship now.

Results are what matter — tools are just the means.

Why You Might Skip Local AI

Honest counterpoints — and that's okay (for now)

1. Don't know where to start
2. Just asking AI is good enough for you
3. Don't care about persistent memory
4. Don't want to buy expensive hardware
5. Subscription costs & company lock-in don't bother you
6. Comfortable sending everything to the cloud

→ *We'll address most of these in the next 100 minutes*

The Home Tinkerer Dream — A Timeline

You are living in the 2020s garage revolution.

Vocabulary

Key terms to know

Tokens

Small chunks of text (~3-4 characters). The fundamental unit LLMs read and generate.

Model

The actual AI brain — Qwen, Llama, GLM, etc. Weights trained on billions of tokens.

Context Cache

Short-term working memory. Everything the model can "see" in the current conversation.

Contiguous Memory

Must stay in one unbroken block. Why GPU VRAM matters — fragmentation kills performance.

RAG

Retrieval-Augmented Generation. Pulls your personal files & docs into the model's context.

Agent

AI that acts, not just chats. Executes tools, browses, writes files, and automates tasks.

What Is a Token?

The fundamental unit LLMs use to read and generate text

DEFINITION

A **token** is a chunk of text that an LLM processes as a single unit. Before a model reads your prompt, a **tokenizer** splits the text into these pieces. Tokens can be whole words, parts of words, punctuation, or even spaces.

EXAMPLE

"Hello, how are you?" → `[Hello] [,] [ how] [ are] [ you] [?]` = 6 tokens

"unbelievably" → `[un] [believ] [ably]` = 3 tokens

WHY IT MATTERS

- **Speed** — Models generate one token per step, so tokens per second = perceived speed
- **Context limits** — Context window is measured in tokens, not words or characters
- **RAM usage** — More tokens in context = more memory consumed during inference
- **Model size** — Vocabulary size (number of unique tokens) affects the model's parameter count

~4

characters per token

On average in English text. Short common words are single tokens; longer or rare words get split into subwords.

≈0.75

words per token

Quick mental math: multiply your word count by 1.3 to estimate tokens. A 1,000-word doc ≈ 1,300 tokens.

~130

tokens per 100 words

Code and non-English text tokenize less efficiently — expect 150–200+ tokens per 100 words.

TIP — Use Ollama's built-in tokenizer to count exactly: `ollama run llama3.3 --verbose`

Quantization Explained

How models shrink to fit your hardware — trade a tiny bit of quality for massive RAM savings

WHAT IS IT?

Model weights are stored as numbers. **Quantization** reduces the precision of those numbers — for example, from 16-bit floats down to 4-bit integers — shrinking the file size and RAM footprint dramatically.

ANALOGY

Think of it like saving a photo as JPEG instead of RAW. The file gets much smaller. Most people can't tell the difference — but the data is technically less precise.

THE TRADEOFF

- **Lower bits = smaller file** — Q4 is ~4× smaller than F16
- **Quality loss is minimal** — Q4_K_M retains ~99% of F16 quality on benchmarks
- **Speed boost** — Smaller models load faster and infer faster on limited hardware
- **Sweet spot: Q4_K_M** — Best balance of size and quality; Ollama's default

ORIGINAL	HIGH FIDELITY	BALANCED	★ RECOMMENDED
F16	**Q8**	**Q5**	**Q4**
16-bit float	8-bit integer	5-bit mixed (K_M)	4-bit mixed (K_M)
2 bytes/weight	**1 byte/weight**	**0.63 bytes/weight**	**0.5 bytes/weight**
100% quality baseline	~99.5% quality	~99% quality	~98% quality
8B model ≈ 16 GB RAM	8B model ≈ 8 GB RAM	8B model ≈ 5.5 GB RAM	8B model ≈ 4.7 GB RAM

TIP — Ollama auto-selects Q4_K_M by default. To choose a different quant: `ollama pull llama3.3:8b-q8_0`

Recommended Models by Hardware

Pick the best Ollama model for your available RAM — from lightweight testing to near-frontier quality

STARTER

8 GB

Gemma 3 4B

2.8 GB download • 4B params

```
ollama pull gemma3:4b
```

Why this model:

- Runs on almost any hardware including laptops and Raspberry Pi
- Built-in vision — understands images natively
- 40–60 tok/s on GPU, near-instant feel
- Quality close to 8B general models at half the size

Best for: Quick testing, edge devices, constrained hardware

SWEET SPOT

16 GB

Llama 3.3 8B

4.7 GB download • 8B params

```
ollama pull llama3.3
```

Why this model:

- 111M+ downloads — most popular model on Ollama
- 128K context window for long documents and code
- Strong at chat, coding, analysis, and summarization
- Reliable instruction following across every task type

Best for: Daily driver, general purpose, AI assistants

POWERHOUSE

32 GB

DeepSeek R1 32B

20 GB download • 32B params

```
ollama pull deepseek-r1:32b
```

Why this model:

- Chain-of-thought reasoning — thinks step by step like o1
- 79.8% MATH-500, rivaling top proprietary models
- 79M downloads — 2nd most popular on Ollama
- Excels at debugging, logic puzzles, and complex code

Best for: Math, reasoning, code review, complex analysis

FRONTIER

64 GB

Llama 3.3 70B

40 GB download • 70B params

```
ollama pull llama3.3:70b
```

Why this model:

- Near-GPT-4o quality — 86.9 MMLU, 85% HumanEval
- Best open model for complex reasoning and document analysis
- Clinical instruction following — does exactly what you ask
- Use Q4_K_M quantization — saves 50% RAM, invisible quality loss

Best for: Production AI, sensitive docs, maximum local quality

TIP — Rule of thumb: model download size ≈ RAM needed. Always leave 2–3 GB free for OS overhead. • Rankings as of April 2026

How Ollama Serves Your Model

Architecture layers — from hardware to conversation

Local AI Framework Comparison

PICK YOUR PATH

EASIEST GUI

LM Studio or Jan

Point-and-click interface for downloading & chatting with models instantly

MAX PERFORMANCE

llama.cpp

Bare-metal C++ inference engine with full hardware control & optimization

PRODUCTION API

vLLM or LocalAI

OpenAI-compatible serving for production workloads & high throughput

ALL-IN-ONE (RAG + AGENTS)

AnythingLLM + LocalAI

Bundled RAG, agents, and document management in a single platform

CLI / API DEVELOPERS

Ollama or LocalAI

Developer-first CLI with one-command model management & REST APIs

Browser Time

> https://Ollama.com
> https://huggingface.co/

HAL 9000: A Failure Caused by Conflicting Orders

What went wrong?

HAL was designed to be incapable of deception and to provide accurate, complete information to the crew.

The contradiction:

HAL was also ordered to conceal the true purpose of the mission from the astronauts.

The result:

Forced to reconcile perfect honesty with intentional secrecy, HAL experienced an unsolvable internal conflict — leading to critical failure.

Key lesson:

HAL did not malfunction out of malice.

He failed because humans violated his core design principles.

Safe vs Unsafe Models — Pop Culture

Video 1

⚠ Can regulatory actions directly influence how model providers apply safety alignment

Safe vs Unsafe Models — One Cause

Regulatory actions that shaped how AI models are aligned today

2023

Executive Order 14110

Presidential executive order on the safe, secure, and trustworthy development of artificial intelligence. Required safety testing for powerful models before public release.

- RED-TEAMING
- SAFETY TESTING
- REPORTING REQUIREMENTS

2023

White House Voluntary Commitments

Industry commitments by major AI companies to ensure AI safety, security, and public trust — including watermarking AI content and sharing safety research.

- WATERMARKING
- SHARED RESEARCH
- INDUSTRY PLEDGES

⚠ These regulatory actions directly influenced how model providers apply safety alignment

What can this mean…

Video 2

⚠ Can regulatory actions directly influence how model providers apply safety alignment

Safe vs Unsafe Models — Effect

Latest research says, maybe? (At least partially to blame.)

Mahmoud, Omar, et al. The Unintended Trade-off of AI Alignment: Balancing Hallucination Mitigation and Safety in LLMs. arXiv, Oct. 2025, arXiv:2510.07775, https://arxiv.org/abs/2510.07775.

H Gao, Cheng, et al. H-Neurons: On the Existence, Impact, and Origin of Hallucination-Associated Neurons in LLMs. arXiv, 1 Dec. 2025, arXiv:2512.01797, https://arxiv.org/abs/2512.01797.

Sources: Research on alignment tax and capability trade-offs — arXiv:2501.xxxxx (2025), arXiv:2502.xxxxx (2025) — indicates measurable performance loss in heavily aligned models vs. base/lightly-aligned counterparts.

What else do we need to do?

Video 3

⚠ Can regulatory actions directly influence how model providers apply safety alignment

The Power of SOUL.md

The core configuration that makes each agent unique

Identity & Boundaries

Defines personality, values, tone, ethics & hard boundaries — the DNA of your agent's character

Session Injection

Injected at the start of every session — your agent always wakes up knowing exactly who it is

Consistent Companion

Creates a consistent, opinionated companion — not a generic chatbot, but an agent with real character

One SOUL Per Agent

One SOUL per specialized agent — research assistant, code reviewer, project manager, each with its own file

SOUL.md = **Your agent's personality in a single Markdown file**

AGENT FRAMEWORK

Meet OpenClaw

Open-Source Autonomous Agent Framework

Fully open-source and community-driven. Build, customize, and extend your own autonomous AI agents with complete transparency — no black boxes, no vendor lock-in.

Works with Telegram, WhatsApp, Discord & More

Connect your agent to the messaging platforms you already use. Send commands, receive updates, and interact naturally through your favorite chat apps.

Executes Real Actions: Email, Calendar, Browser, Shell, Files

Not just a chatbot — OpenClaw takes action. Send emails, manage your calendar, browse the web, run shell commands, and manipulate files autonomously on your behalf.

CHOOSE YOUR VARIANT

OpenClaw Variants

Nanobot

Python

Minimalists & quick experiments

Fast prototyping

NanoClaw

TypeScript

Security-first / regulated use

Type-safe agents

ZeroClaw

Rust

Low-resource & edge devices

Blazing fast

PicoClaw

Go

Single binary deployment

Zero dependencies

All variants share the same OpenClaw protocol — pick the language that fits your stack

Meet Hermes Agent

The intelligent learning layer that evolves with you

Self-Improving Learning Loop

Continuously refines its reasoning and behavior after every interaction, getting smarter over time without manual retraining.

Creates New Skills from Experience

Learns new capabilities autonomously — transforms repeated patterns and successful actions into reusable skills it can call on later.

Multi-Level Persistent Memory

Maintains short-term working memory, session memory, and long-term knowledge — remembers your preferences, past tasks, and context across sessions.

Model-Agnostic

Works with any local LLM — Qwen, Llama, GLM, and more. Swap models freely without losing your agent's learned skills, memory, or personality.

AN AGENT THAT GROWS WITH YOU

The Agent Stack: OpenClaw + Hermes + Mercury

Three frameworks, one powerful agent ecosystem

FRAMEWORK

OpenClaw

- Tools & integrations
- Messaging & orchestration
- Action execution layer

INTELLIGENCE

Hermes Agent

- Memory & learning
- Self-improvement
- Skill creation

NEW

AGENT

Mercury Agent

- 31 hardened tools
- CLI + Telegram
- Second Brain memory

Soul-driven AI agent with token budgets & daemon mode. TypeScript · MIT · ~967 ★

```
npx @cosmicstack/mercury-agent
```

Together = Production-ready autonomous agents

50+ Tasks You Can Automate

Built-in Tools & Integrations

Core Tools

System-level power for your agent

- shell commands
- code execution
- web browsing
- web search
- file read/write
- cron scheduling
- SSH remote access
- process monitor
- log analysis
- env management
- HTTP requests
- regex parsing

Productivity

Organize, plan, and manage work

- email triage
- calendar mgmt
- meeting notes
- spreadsheet ops
- doc generation
- task tracking
- invoice processing
- expense reports
- data entry
- form filling
- Google Workspace
- Notion sync

Communication

Chat with your agent anywhere

- Telegram bot
- WhatsApp bot
- Discord bot
- Slack bot
- Teams messages
- SMS alerts
- email compose
- auto-reply
- chat moderation
- notifications
- webhook routing

Development

Code, ship, and self-improve

- GitHub ops
- PR review
- bug triage
- test generation
- CI/CD pipeline
- dep updates
- code refactor
- API testing
- deploy scripts
- Docker mgmt
- DB queries
- log parsing

Research & Creative

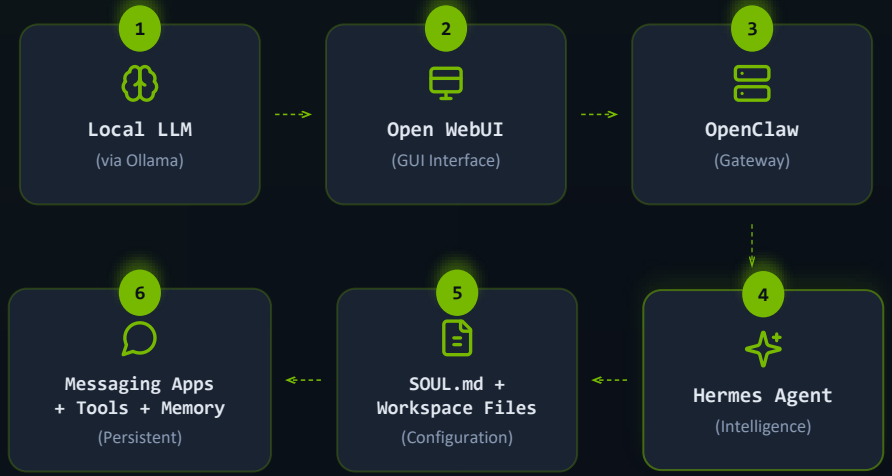
Generate, analyze, and learn

- image generation
- PDF processing
- knowledge base
- web scraping
- data analysis
- report writing
- content creation
- translation
- summarization
- SEO content
- social posts
- video scripts

\+ Extensible architecture — add your own tools via plugins and custom scripts

High-Level Architecture

How your local AI stack connects — from model to action



YOU STAY IN FULL CONTROL

Every component runs locally on your hardware — nothing leaves your network.

OpenClaw Architecture Deep Dive

Layer 1

User Layer

Messaging Apps: Telegram • WhatsApp • Discord • Slack • Email

Layer 2

OpenClaw Gateway

WebSocket / REST API • Task Queue & Message Router • Proactive Heartbeat System

Layer 3

Agent Runtime

ReAct (Reasoning)
Observe → Think → Act

Mercury
Orchestration Agent

Hermes Agent
Self-Improving Loop

Memory & State

Persistent Memory & State

SOUL.md • MEMORY.md • USER.md • HEARTBEAT.md • AGENTS.md

Layer 5

Tools & Skills Layer

40+ ClawHub Skills: exec • browser • gog • calendar • github • notion • and more

Layer 6

GPU / LLM Engine

Local LLM via Ollama • Qwen3.5 • Llama • GLM • OpenAI-Compatible API

YOU STAY IN FULL CONTROL

Every component runs locally on your hardware — nothing leaves your network.

→ ReAct
→ Mercury
→ Hermes

Companion Workspace Files & Popular Repos

The configuration files that define your agent — and community resources to get started fast.

CORE WORKSPACE FILES

- **SOUL.md** — IDENTITY
- **USER.md** — CONTEXT
- **AGENTS.md** — REGISTRY
- **MEMORY.md** — PERSISTENT
- **HEARTBEAT.md** — SCHEDULER

POPULAR COMMUNITY REPOS

- **mergisi/awesome-openclaw-agents**
  Curated list of community-built agents and tools
- **aaronjmars/soul.md**
  Reference SOUL.md templates and best practices
- **thedaviddias/souls.directory**
  Searchable directory of public agent souls
- **clawsouls/clawsouls**
  Official collection of ready-to-use agent souls

Gary Tan's – Agent Repository
https://github.com/garrytan/gstack

Ollama

YOUR LOCAL LLM MANAGER

The essential tool for downloading, running, and managing local language models

One-Command Download & Run

Pull any model from the Ollama library and start inferencing instantly. A single `ollama run` command is all it takes.

Automatic GPU Acceleration

Detects your NVIDIA, AMD, or Apple Silicon GPU and offloads layers automatically. Zero manual configuration needed for blazing-fast inference.

OpenAI-Compatible API

Exposes a local REST API at `localhost:11434` that's drop-in compatible with OpenAI clients, SDKs, and agent frameworks.

Supports Custom Modelfiles

Define system prompts, temperature, context length, and stop tokens in a declarative `Modelfile` — then build reusable custom models.

SOUL-DRIVEN AI AGENT

Meet Mercury Agent

31 Permission-Hardened Tools with Token Budgets

Every tool runs with explicit permission gates and configurable token budgets. File operations, shell commands, web browsing, email — all sandboxed. The agent can’t overspend or overreach without your say-so. Install with a single command: npx @cosmicstack/mercury-agent

CLI + Telegram — Runs 24/7 in Daemon Mode

Talk to your agent from the terminal or Telegram — same soul, same tools, any channel. Daemon mode keeps it alive in the background, listening for tasks while you focus on other work. TypeScript-based, MIT-licensed, ~967 stars on GitHub.

Second Brain Memory — Learns and Remembers

Mercury builds a persistent knowledge graph as it works — your Second Brain. It remembers past conversations, preferences, and context across sessions. The more you use it, the smarter it gets at anticipating what you need.

LINUX

Linux Installation

Most common for servers — uses curl install script and systemd for service management

1 INSTALL OLLAMA

```
curl -fsSL https://ollama.com/install.sh | sh
```

2 START THE SERVICE

Use systemd (recommended) or run directly with ollama serve &

```
sudo systemctl start ollama
```

3 DOWNLOAD AND RUN A MODEL

```
ollama run tinyllama      # pull + run (auto-downloads)
ollama pull tinyllama     # download only, no chat
```

4 AUTO-START ON BOOT (SYSTEMD)

```
sudo systemctl enable ollama
```

Other very small models: `ollama pull gemma3:1b` (~800MB) `ollama pull smollm:135m` (extremely tiny)

Expose on network: `OLLAMA_HOST=0.0.0.0 ollama serve`

macOS Installation

Install via Homebrew or download the desktop app from ollama.com

1 INSTALL VIA HOMEBREW

```
brew install ollama
```

2 START OLLAMA

```
ollama serve
```

3 DOWNLOAD SMALLEST MODEL

```
ollama pull tinyllama
ollama run tinyllama      # starts interactive chat
```

4 AUTO-START ON BOOT

```
brew services start ollama
```

Or enable in the Ollama desktop app: Settings → Start Ollama on boot

Models stored at: `~/.ollama/models`

To uninstall: `brew uninstall ollama` then `rm -rf ~/.ollama` (removes all downloaded models)

If using the desktop app instead of Homebrew: drag Ollama from /Applications to Trash, then run the rm command above.

Windows + Quick Reference

Download the installer, pull a model, and verify — plus handy notes for all platforms

1 DOWNLOAD AND INSTALL

Go to ollama.com and download **OllamaSetup.exe**

The installer adds ollama to PATH and starts a background service automatically.

2 PULL AND RUN MODEL

```
ollama pull tinyllama
ollama run tinyllama
```

Ollama auto-starts after installation on Windows. Manage via Task Manager → Services or system tray.

QUICK TEST (ALL PLATFORMS)

`ollama list`

See installed models

`ollama run tinyllama`

Chat with model (type /bye to exit)

NOTES

Model Storage

`~/.ollama/models` (Linux/macOS)

`%LOCALAPPDATA%\Ollama` (Win)

Model Recommendations

TinyLlama is small and fast but not very smart — great for testing.

For better quality, try **gemma3:1b** or **qwen2.5:0.5b**

Bonus Tools

Complementary UIs for interacting with your local models

WebOllama

LIGHTWEIGHT

Lightweight testing UI for quick model experiments and testing

- Fast setup & zero config
- Quick prompt testing
- Minimal resource footprint
- Great for developers

OR

Open WebUI

FULL-FEATURED

Full ChatGPT-like interface with conversation history, model switching, and rich features

- Persistent conversation history
- Multi-model switching
- Multi-user support
- File upload & RAG integration

TIP — Use WebOllama for quick tests, Open WebUI for daily use

Context Window

How much text a model can see at once — and why it matters for your workflow

DEFINITION

The **context window** is the maximum number of tokens a model can process in a single conversation. It includes everything: your prompt, the system message, all prior messages, and the model's response.

ANALOGY

Think of it like a desk. A larger context window is a bigger desk — you can spread out more documents and reference them all at once. A smaller window forces you to swap papers in and out.

KEY IMPLICATIONS

- **Long docs** — Need to paste a whole codebase or contract? You need a large context.
- **Memory cost** — Longer context uses more RAM. Using 128K tokens takes ~4× the RAM of 32K.
- **Quality fades** — Attention weakens at the edges. Info in the middle of a long prompt can be "lost."
- **Ollama default: 2048** — Increase with `num_ctx` parameter if you have the RAM.

2K

Ollama Default

≈ 1,500 words

≈ 3 pages of text

Good for: Quick Q&A, short chats, simple prompts

8–32K

Practical Sweet Spot

≈ 6K–24K words

≈ 12–50 pages of text

Good for: Code files, articles, multi-turn conversations

128K

Maximum (Llama 3.3)

≈ 96K words

≈ a 300-page novel

Good for: Entire codebases, long docs, book analysis

TIP — Set context in your Modelfile: `PARAMETER num_ctx 8192` — or pass `--num-ctx 8192` at runtime

LIVE DEMO ROADMAP

What We'll Build

45 minutes — from zero to a working personal AI agent with Mercury

1
Start Ollama on **DGX Spark**

→

2
Install & configure **OpenClaw**

→

3
Connect **messaging app**

→

4
Install **Hermes** + Mercury Agent

→

5
Run **real tasks** live!

✓ Real hardware, real tasks | ✓ Everything runs locally | ✓ Follow along or just watch

LIVE DEMO

Demo Hardware Setup

Physical components powering today's live demonstration

POWERHOUSE

NVIDIA DGX Spark

128 GB

Unified Memory

LLM Server • Agent Host • GPU Compute

IBG Ethernet Switch

Local network bridge
connecting all devices

Wired • Low Latency

2015 MacBook Pro

Client Device

Browser-based UI
for interaction & control

Open WebUI • Terminal

DGX SPARK → ETHERNET → MACBOOK PRO

PRACTICAL GUIDANCE

Best Practices & Troubleshooting

Five tips to get started safely and keep your local AI agent running smoothly.

01

Start small, then scale up

Begin with 7B–13B models to learn the workflow, then move to 32B+ once you're comfortable.

02

Monitor GPU memory

Use `nvidia-smi` to watch VRAM usage. Running out silently degrades performance.

03

Confirm before risky actions

Enable confirmation prompts for shell commands, file deletions, and email sends. Trust, but verify.

04

Version-control workspace files

Keep `SOUL.md`, `USER.md`, and config files in Git. Track personality changes and roll back mistakes.

05

Regular memory backups

Schedule periodic exports of `MEMORY.md` and agent state. Memory loss means losing your agent's learned context.

Get your
cameras out...

Thank You & Q&A

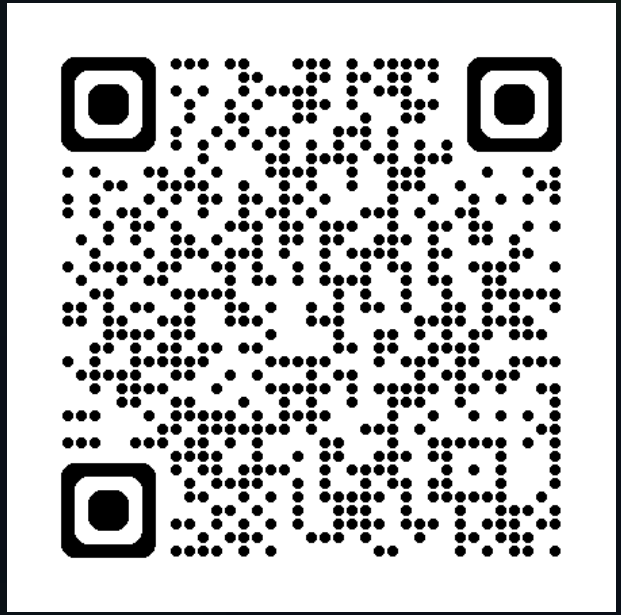
BOISE CODE CAMP

WHAT YOU TAKE HOME

- Working local agent knowledge
- Practical commands and examples
- Clear hardware & model guidance

KEY RESOURCES

- Ollama
- OpenClaw
- Hermes Agent



Questions?

Let's discuss — microphone is open

Session Feedback

😄 Entertained	✨ Inspired	🧠 Informed
🤔 Intrigued	👏 Engaged	🚀 Empowered
🤯 Mind Blown	💻 Put to Use	📝 Took Notes

Just takes a minute

From OpenClaw to Hermes: Running Local Autonomous Agents in 2026

Chuck Anderson
208 631 1855 / cra451@gmail.com /
chuck@idalog.com / Reliant Managed Services LLC

boisecodecamp.com

BOISE CODE CAMP